

# PROBLEM B: PIXEL ART

*International Computer Science Competition 2026 — Qualification Round*

## 1. Algorithmic Approach

---

The objective is to populate an  $N \times N$  grid structure based on geometric and mathematical indexing constraints rather than relying on conditional overrides or multipass matrix manipulation.

**Checkerboard Pattern:** To ensure that alternating fields match standard grid-alternation sequences while guaranteeing that the absolute top-left cell  $(0,0)$  evaluates directly to  $0$ , we implement a check on the linear summation of the current multi-dimensional loop indices. For any cell located at coordinate  $(i, j)$ , the matrix cell is assigned the evaluation of  $(i + j) \% 2$ .

**Diamond Shape:** A bounding diamond grid within an odd-dimensioned allocation matrix represents an isometric distance bounds constraint. By establishing the exact matrix spatial center point at `center = N // 2`, we apply the 2D Manhattan distance formulation. A cell is enabled (assigned 1) if and only if the absolute index offset sum fulfills the relation:  $\text{abs}(i - \text{center}) + \text{abs}(j - \text{center}) \leq \text{center}$ .

## 2. Python 3.9 Source Code

---

```
def generate_shape(n: int, shape: str) -> list:
    # Initialize an N x N matrix populated with default zero-states
    grid = [[0] * n for _ in range(n)]

    if shape == "checkerboard":
        for i in range(n):
            for j in range(n):
                # Parity of the combined coordinate indexes dictates spatial toggling
                grid[i][j] = (i + j) % 2

    elif shape == "diamond":
        center = n // 2
        for i in range(n):
            for j in range(n):
                # Mathematical Manhattan distance threshold determines geometry boundary
                if abs(i - center) + abs(j - center) <= center:
                    grid[i][j] = 1

    return grid
```

```
if __name__ == "__main__":  
    # Structural local execution entry point  
    pass
```